

ES modules

What Are ES Modules?

ES Modules (ESM) are the official JavaScript module system introduced in ES6 (2015).

Basic Syntax

Exporting

// math.js

```
export const PI = 3.14;
```

```
export function add(a, b) {  
  return a + b;  
}
```

Importing

// app.js

```
import { PI, add } from "./math.js";
```

```
console.log(add(2, 3));
```

Default Export

// logger.js

```
export default function log(msg) {  
  console.log(msg);  
}
```

// app.js

```
import log from "./logger.js";
```

If exported default=direct import no {}

Normal export {} import

The Most Important Concept — Static Structure

That means:

- Imports are resolved before execution
- You cannot conditionally import like:

```
if (true) {  
import { something } from "./file.js"; ✗  
}
```

Because imports must be at the top level.

Why?

Because the JS engine builds a dependency graph before running any code.

How ESM Actually Executes (Deep Dive)

When you run:

```
import { x } from "./a.js";
```

JS does 3 phases:

- ◆ 1. Parsing Phase
 - Build module dependency graph
 - Identify all imports/exports
- ◆ 2. Instantiation Phase
 - Create memory bindings

- Allocate variables
- Link imports to exports
- ◆ 3. Evaluation Phase
 - Execute the code

This is why:

`console.log(x);` // works even if x defined later in file

Because bindings exist before execution.

Live Binding

When you import something from a module, you don't get a copy — you get a live reference to the original variable.

So if the exporting module changes the value, the importing module sees the updated value automatically.

counter.js

```
export let count = 0;
```

```
export function increment() {  
  count++;  
}
```

app.js

```
import { count, increment } from "./counter.js";
```

```
console.log(count); // 0
```

```
increment();
```

```
console.log(count); // 1 ☒ updated
```

Notice:

We never reassigned count in app.js.

You cannot reassign an imported binding.

```
import { count } from "./counter.js";
```

```
count = 10; ❌ Error
```

Why?

Because:

- The binding is live
- But it's read-only from the importing side

You can observe it.

You can't mutate it directly.

Only the exporting module controls it.

Module Scope

Each module has its own scope.

```
// a.js
```

```
const secret = 123;
```

```
// b.js
```

```
console.log(secret); // ❌ Error
```

Nothing leaks to global.

Unless explicitly attached to window.

Top-Level Await

In ESM:

```
const data = await fetch("https://api.com/data");
```

This pauses module execution until resolved.

Powerful for:

- Config loading

- Database connections
- Startup logic

Circular Dependencies

ESM handles circular dependencies better than CommonJS because of:

- Live bindings
- Separate instantiation phase

But still tricky:

```
// a.js
import { b } from "./b.js";
export const a = "A";

// b.js
import { a } from "./a.js";
export const b = "B";
```

Bindings exist, but values may be undefined during evaluation.

This is a senior-level debugging area.

Node.js + ESM

To use ESM in Node:

Option 1: package.json

```
{
  "type": "module"
}
```

Option 2: .mjs extension

file.mjs

Browser Usage

```
<script type="module" src="app.js"></script>
```

What Is Tree Shaking?

Tree shaking is a build-time optimization that removes unused code from your final bundle.

Why ES Modules Enable Tree Shaking

Because:

- import is static
- Bundlers know exactly what is imported
- No runtime dynamic require guessing

Who Performs Tree Shaking?

Bundlers like:

- Webpack
- Rollup
- Vite
- esbuild

The JavaScript engine itself does NOT tree shake.
It happens during build time.